



안양대학교 일반대학원 · 미세먼지관리전공

CLAUDE CODE

AI와 함께 미세먼지 연구 코드 작성하기

Session 3 — AI 보조 코딩의 실전

전제 Session 1-2 완료 (GitHub 저장소)

도구 Claude Code (터미널 기반)

시간 2시간 (Zoom)

개념 이해

Claude Code란? — 터미널 속의 AI 연구 보조원



Claude.ai (웹/앱)

- ▶ 대화형 인터페이스
- ▶ 질문하면 답을 줌
- ▶ 코드를 복사해서 써야 함
- ▶ 파일 직접 수정 불가



Claude Code (터미널)

- ▶ 내 프로젝트 폴더 안에서 실행
- ▶ 파일을 직접 읽고 수정
- ▶ 코드 작성 → 테스트 → 수정 자동화
- ▶ Git 명령어도 대신 실행



연구자에게 Claude Code = AI 공동연구자

"이 PM2.5 데이터에서 이상값 제거하고 계절별 평균 계산하는 코드 짜줘"라고 말하면, 실제로 파일을 만들어줍니다.

실습 준비

Claude Code 설치 (5분)

1

Node.js 설치 확인

node.js.org → LTS 버전 설치 (이미 있으면 생략)

2

Claude Code 설치

터미널에서 아래 명령어 실행

3

연구 폴더에서 시작

pm25-thesis 폴더로 이동 후 claude 실행

Claude Code 전역 설치

```
npm install -g @anthropic-ai/claude-code
```

pm25-thesis 폴더로 이동

```
cd pm25-thesis
```

Claude Code 시작 (처음 실행 시 로그인 안내)

```
claude
```

```
✓ Welcome to Claude Code! Let's build something great.
```

```
>
```

copy

실습 1

PM2.5 분석 코드 첫 번째 생성

 CLAUDE CODE 프롬프트 예시

나는 안양대 미세먼지관리전공 석사생이야.

현재 pm25-thesis/analysis 폴더에 작업 중이고,
AirKorea에서 받은 PM2.5 일별 측정 데이터 (날짜, 측정소명, PM2.5 농도)를 분석하려고 해.

다음 기능을 하는 Python 스크립트를 **analysis/pm25_analysis.py**로 만들어줘:

1. CSV 파일 읽기 (pandas)
2. 결측값(-999, NaN) 제거
3. 봄/여름/가을/겨울 계절별 평균 PM2.5 계산
4. 계절별 박스플롯 저장 (figures/seasonal_boxplot.png)

✅ Claude Code는 파일을 직접 생성하고, 코드를 실행해보고, 오류가 나면 스스로 수정합니다.

이전 (tab)

핵심 스킬

좋은 프롬프트의 3가지 원칙



① 맥락 제공

- ▶ 나의 역할 (미세먼지 연구 석사생)
- ▶ 데이터 구조 설명
- ▶ 파일 경로 명시
- ▶ 사용 라이브러리 지정



② 구체적 요구

- ▶ 번호로 기능 나열
- ▶ 입력/출력 명확히
- ▶ 파일명 지정
- ▶ 그래프 형식 지정



③ 반복 개선

- ▶ "x축 폰트 크기 14pt로"
- ▶ "WHO 기준선 35 μ g/m³ 추가"
- ▶ "한국어 범례로 바꿔줘"
- ▶ 한 번에 완벽하지 않아도 됨

💡 나쁜 프롬프트: "PM2.5 분석해줘" → 좋은 프롬프트: "AirKorea 서울 측정소 2023년 일평균 PM2.5 데이터에서 월별 초과일수(35 μ g/m³ 이상)를 계산하고 막대그래프로 저장해줘"

프롬프트 템플릿

미세먼지관리전공 수업별 프롬프트



대기확산모델링 이론

"AERMOD 출력 파일(.dat)을 읽어서 지점별 PM2.5 최대 농도 분포를 등고선 지도로 그려줘. x축은 동서 거리(m), y축은 남북 거리(m)"



파이썬 기반 환경 통계분석

"PM2.5와 기상인자(온도, 습도, 풍속) 간의 다중회귀분석을 해줘. 상관계수 행렬 히트맵과 회귀계수 표도 같이 저장해줘"



AI기반 미세먼지 예측 및 관리

"scikit-learn으로 Random Forest 모델 만들어서 내일 PM2.5 예측해줘. 특성 중요도 막대그래프, 예측 vs 실제 산점도 저장"



비용편익 분석실습

"PM2.5 저감 정책 시나리오별 비용편익 비교 표와 막대그래프 만들어줘. 엑셀 파일로도 저장해줘 (openpyxl)"

워크플로우

Claude Code + Git 통합 워크플로우

Claude Code 터미널 안에서 직접 Git 명령 가능!

copy

> "현재 저장소 상태 확인하고, 수정된 파일들을 의미있는 커밋 메시지로 커밋해줘"

Running: git status

Modified: analysis/pm25_analysis.py, figures/seasonal_boxplot.png

Running: git add analysis/pm25_analysis.py figures/seasonal_boxplot.png

Running: git commit -m "파이썬 통계분석: PM2.5 계절별 박스플롯 및 기초통계 완성"

> "새 브랜치 experiment/ml-prediction 만들고 거기서 Random Forest 예측 코드 시작해줘"

Running: git checkout -b experiment/ml-prediction

Creating: analysis/ml_pm25_prediction.py

 Claude Code는 코드 작성 + Git 관리를 동시에! 코딩에만 집중하면 됩니다.

활용 전략

코드 이해 — 모르는 코드도 설명받기

 설명 요청 프롬프트

"analysis/pm25_analysis.py의 **seasonal_average** 함수를 한 줄씩 설명해줘.
특히 **groupby**와 **resample**이 뭔지, 석사 수준으로 이해할 수 있게."

 논문 작성 연계 프롬프트

"이 분석 결과(seasonal_stats.csv)를 바탕으로
논문 3장 '결과 및 고찰' 섹션에 쓸 수 있는 결과 요약 문단을 한국어로 작성해줘.
수치는 실제 데이터 기반으로, 환경공학 논문 스타일로."

💡 코드를 이해해야 논문에서 방법론 설명 가능 — Claude Code로 **코드와 논문을 동시에** 작성합니다.

실습 2

오류 발생 시 — AI와 디버깅하기

```
# Python 실행 중 오류 발생 시
Traceback (most recent call last):
  File "analysis/pm25_analysis.py", line 23, in <module>
    df['date'] = pd.to_datetime(df['date'], format='%Y%m%d')
ValueError: time data '2023/01/01' does not match format '%Y%m%d'
```

copy

Claude Code에 그냥 붙여넣기!

> "위 오류가 났어. 원인이 뭔지 설명하고 고쳐줘"

→ 날짜 형식이 '20230101'이 아닌 '2023/01/01'로 되어 있어요.

→ format='%Y/%m/%d' 또는 format='mixed'로 수정하겠습니다.

😓 오류 메시지를 그대로 붙여넣으면 됩니다. "왜 안 돼?"라고 물어도 됩니다.

오류를 무서워하지 마세요 — 이제 AI가 같이 고쳐줍니다.

실전 예시

Claude Code로 만드는 논문급 분석 파이프라인

1

 데이터 수집 & 정제

"AirKorea CSV를 읽어 결측값 제거, -999 필터링, 날짜 인덱스 설정하는 data_cleaning.py 만들어줘"

2

 기초 통계 분석

"계절별, 월별, 요일별 PM2.5 기초통계 (평균, 중앙값, 표준편차) 테이블과 그래프 생성"

3

 기상 상관관계 분석

"기상 데이터와 합쳐서 다중회귀분석, 상관계수 히트맵, VIF 다중공선성 검증"

4

 공간 분포 시각화

"측정소 위도/경도 데이터 활용해서 folium 인터랙티브 지도에 PM2.5 농도 히트맵 그리기"

수업 연계

Cloud 활용 Big data 수업과 연계



Cloud 활용 Big data 관리론

- ▶ AWS S3에 PM2.5 데이터 저장 코드
- ▶ Google Colab에서 대용량 분석
- ▶ GitHub Actions로 자동 데이터 갱신
- ▶ Claude Code가 AWS CLI 명령어 생성



SMART 측정 및 분석 실습

- ▶ 센서 데이터 실시간 수신 코드
- ▶ 이상값 자동 감지 알고리즘
- ▶ SMART 측정 결과 시각화 대시보드
- ▶ Claude Code가 Raspberry Pi 코드도!



Session 1-2 (Git/GitHub) + Session 3 (Claude Code) 통합:

Claude Code로 코드 작성 → Git으로 버전 관리 → GitHub에서 교수님 리뷰 → 논문 완성

주의 사항

AI 코딩 도구 사용 시 반드시 알아야 할 것



코드를 이해해야 한다

AI가 만든 코드도 **내가 이해하고 설명할 수 있어야** 논문에 쓸 수 있습니다. "AI가 짜줬어요"는 논문 방법론이 아닙니다.



결과를 검증해야 한다

AI가 계산한 통계값, 그래프가 **실제로 맞는지 직접 확인**하세요. 환경공학 도메인 지식으로 결과의 타당성 판단이 필요합니다.



데이터 보안

Claude Code에 민감한 개인정보나 미공개 연구 데이터를 직접 붙여넣지 마세요. 샘플 데이터로 테스트 후 실제 데이터 적용



인용 및 출처

AI 생성 코드는 Git으로 관리하고, 논문에서 "Python 3.x, pandas, matplotlib 사용"처럼 **도구를 명시**하세요.

Claude Code 프롬프트 치트시트

===== 코드 생성 =====

"[파일경로]에 [기능]을 하는 Python 코드 만들어줘"

"[라이브러리] 사용해서 [데이터]를 [형식]으로 분석해줘"

===== 코드 수정 =====

"[함수명] 함수에 [기능] 추가해줘"

"그래프 [옵션] 수정해줘: [구체적 내용]"

===== 오류 해결 =====

"이 오류 고쳐줘: [오류 메시지]"

"왜 이 결과가 나왔는지 설명해줘"

===== 논문 연계 =====

"이 분석 결과로 논문 [섹션]에 쓸 문단 작성해줘"

"이 코드 방법론을 학술 논문 스타일로 설명해줘"

===== Git 연계 =====

"변경된 파일들 의미있는 메시지로 커밋해줘"

"새 브랜치 [이름] 만들고 거기서 시작해줘"

copy

세션 요약

Session 3 핵심 정리



Claude Code 핵심

- ▶ `npm install -g @anthropic-ai/claude-code`
- ▶ 프로젝트 폴더에서 claude 실행
- ▶ 파일 직접 생성/수정
- ▶ Git 명령도 대신 실행



좋은 프롬프트

- ▶ 맥락 + 구체적 요구
- ▶ 파일명/경로 명시
- ▶ 반복 개선 OK
- ▶ 코드 이해 필수!



다음 세션 예고

- ▶ OpenAI Codex
- ▶ GitHub Copilot
- ▶ 두 AI 도구 비교
- ▶ 더 고급 AI 활용법

과제

Session 3 과제

과제 (다음 세션 전까지)

1. Claude Code로 **pm25_analysis.py** 기본 버전 생성

샘플 데이터(직접 만든 임의 CSV)로도 OK — 코드 구조가 중요

2. 최소 4가지 기능 포함: **데이터 로드 / 결측값 처리 / 계절별 통계 / 그래프 1개**

파이썬 기반 환경 통계분석 수업 내용 연계 권장

3. **논문 요약 카드 2개**를 docs/paper_summaries/에 Markdown으로 작성

연구문제, 데이터, 방법, 결과, 한계, 내 논문에서 쓸 위치를 반드시 포함

4. 작업 완료 후 **의미있는 커밋 3개 이상** + GitHub push

커밋 메시지: "Claude Code 지원: [기능명] 추가" 형식 사용

5. README.md에 **코드 실행 방법** 추가 (requirements.txt 포함)

예: pip install pandas matplotlib seaborn

6. **보너스**: AI가 만든 코드 한 함수를 직접 설명해보기 (다음 세션 발표)